

C-Bounded Powersort Pseudocodes

Théo Araújo Magalhães

April 14, 2025

TO-DOS:

1. Os comentários para cada algoritmo precisam estar de acordo com as definições usadas no primeiro esboço do artigo (powersort.tex) e precisam ser revisados para fazerem sentido, alguns ainda estão bem abstratos e/ou não descrevem corretamente os algoritmos.
2. Tanto para esse pdf quanto para o anterior, ou seja, para o artigo como um todo, o nome do professor Victor se enquadra como autor? Imagino que sim. Essa pergunta é meio boba.
3. Código para a estrutura de dados que iremos usar.
4. Reestruturar o vai e volta.

Planejamento da estrutura: Guardaremos o início da primeira run, seu fim e seu power com a run anterior (se houver), além disso, cada elemento seguinte será: o fim da run e o power da run com a anterior. run e não teremos um elemento sentinela. O início de uma run baseia-se no fim da anterior + 1. Assim que incluímos um elemento que estoura o tamanho da pilha, temos que esquecer o primeiro elemento e ajustar o novo primeiro elemento para também guardar o seu início. (cada pop ou insert decrementa ou aumenta o tamanho)

Como vamos fazer scans da esquerda para a direita e da direita para a esquerda, precisamos alternar entre somar um no fim de uma run para achar o começo da próxima e diminuir um. $\leftarrow$ pensar nisso

Comentário do professor: “Só um detalhe de notação, usa aqui (A, s_2, n) . $A[s_2]$ é o valor e não o índice.”

Resposta: Fiz desse jeito porque é a forma que eles escreveram no artigo, mas achei bem estranho também, depois vou mudar.

Introduction: We are very smart and we know things, this is the complaints we have for things that weren't done by us and how we would have made them better.

1 Powersort Versions:

1.1 Standard Powersort

Commentary on Wild and Munro's version: A single pass algorithm that sorts the entire array.

Algorithm 1 Powersort($A[1..n]$) by Wild and Munro:

```
1  $X :=$  stack of runs // capacity  $\lfloor \lg n \rfloor + 1$ 
2  $P :=$  stack of powers // capacity  $\lfloor \lg n \rfloor + 1$ 
3  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4 while  $e_1 < n$ 
5    $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$  //  $A[s_2..e_2]$  next run
6    $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  //  $P_j$  for node  $j$  between  $A[s_1..e_1]$ 
   and  $A[s_2..e_2]$ 
7   while  $P.\text{top}() > p$  // previous merge deeper in tree than current
8      $P.\text{pop}()$  //  $\rightarrow$  merge and replace run  $A[s_1..e_1]$  by result
9      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
10   $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
11   $s_1 := s_2; e_1 := e_2$ 
12 end while // Now  $A[s_1..s_1]$  is the rightmost run
13 while  $\neg X.\text{empty}()$ 
14    $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
```

1.2 First variation of C-Bounded Powersort (only goes right)

Commentary on our first version: This algorithm performs multiple passes on the array, processing all subtrees of height $C-1$ with each pass. We remind the reader of how Powersort works: It is implicitly a complete binary tree of size $\log n$. Therefore, the max amount of passes needed to sort the array is:

$$\text{N}^\circ \text{ of scans} \leq \frac{\log n}{C-1} \quad (1)$$

Algorithm 2 C-Bounded-Powersort 1($A[1..n]$, C):

```

1  $X :=$  C-bounded stack of runs // capacity  $C$ 
2  $P :=$  C-bounded stack of powers // capacity  $C$ 
3  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4 while  $e_1 < n$  // This while can be tweaked to use max amount of passes
5   while  $e_1 < n$  // A single pass goes until the end of the array
6      $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$ 
7      $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  // Using our node power algorithm
8     while  $P.\text{top}() > p$  // We follow the same merge policy
9        $P.\text{pop}()$ 
10     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
11     $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
12     $s_1 := s_2; e_1 := e_2$ 
13  end while
14  while  $\neg X.\text{empty}()$ 
15     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
16    // We're using a limited stack, so we need to check for another pass
17     $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$  // If  $e_1 = \text{end}$ , the array is sorted
18 end while

```

After first modifications:

1.3 Second variation of C-Bounded Powersort (goes left and right!)

Commentary on our second version: This works rather similarly to our first version. The main difference is the fact that, once we reach the end of the array, instead of restarting our scan from the beginning, we scan, starting from the end, to the left, until we reach the beginning of the array, and, thus, we repeat the scan-right process. This is slightly better than the first version, given that it takes advantage of runs still being in cache instead of restarting from the beginning. **Comentário sobre X.size ≤ 1:** Ao invés de termos uma stack X e P, teremos somente uma Stack guardando power e runs. Assim, caso tenhamos uma run dentro da stack após o final de um scan, ela não estará com o 'power' correto quando o próximo scan iniciar. Isso significa que teríamos que dar um pop nessa pilha e tratá-la como vazia. Podemos fazer um $s_1 \leftarrow \text{top}$, visto que a run a ser achada por $e_1 := \text{ExtendRunSide}$ depois de um scan vai ser sempre a run resultante de todos os merges finais (será?)

Algorithm 3 C-Bounded-Powersort 1($A[1..n]$, C):

```

1   $X :=$  C-bounded stack of runs
2   $P :=$  C-bounded stack of powers
3  while true
4     $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
5    while  $e_1 < n$ 
6       $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$ 
7       $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$ 
8      while  $P.\text{top}() > p$ 
9         $P.\text{pop}()$ 
10      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
11      $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
12      $s_1 := s_2; e_1 := e_2$ 
13   end while
14   while  $X.\text{size} > 1$ 
15      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
16    $e_1 = \text{ExtendRunLeft}(A[n], 1)$  // Now, we're scanning from right to left!
17   if  $e_1 == 1$ 
18     break // If we could extend until the beginning, our run is the entire array.
19   while  $e_1 > 1$  // We're scanning from n down to 1
20      $s_2 := e_1 - 1; e_2 := \text{ExtendRunLeft}(A[s_2], 1)$ 
21      $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$ 
22     while  $P.\text{top}() > p$ 
23        $P.\text{pop}()$ 
24      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
25      $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 

```

```

26          $s_1 := s_2; e_1 := e_2$ 
27     end while
28     while  $X.size > 1$ 
29          $(s_1, e_1) := \text{Merge}(X.pop(), A[s_1..e_1])$ 
30          $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$  // Now, we're scanning from left
to right!
31         if  $e_1 == n$ 
32             break // If we could extend until the end, our run is the entire array.

```

2 NodePower versions:

2.1 Standard Nodepower:

Commentary on Wild and Munro's Node Power: This is so silly and expensive!

Algorithm 4 NodePower(s_1, e_1, s_2, e_2, n):

```

1   $n_1 := e_1 - s_1 + 1; n_2 := e_2 - s_2 + 1; \ell := 0$ 
2   $a := (s_1 + n_1/2 - 1)/n; b := (s_2 + n_2/2 - 1)/n$ 
3  while  $\lfloor a \cdot 2^\ell \rfloor == \lfloor b \cdot 2^\ell \rfloor$  do  $\ell := \ell + 1$  end while
4  return ( $\ell$ )

```

2.2 Our Nodepower:

Commentary on our Node Power: Ours is cool and epic. There is a catch, though. Our Node Power doesn't clearly define a Merge Tree, sometimes one run will share the same power between its two neighbors. This can be ignored and simply solved by changing our merge policy to:

$$\mathbf{While} P.top() \geq p \tag{2}$$

instead of:

$$\mathbf{While} P.top() > p \tag{3}$$

Algorithm 4 NodePower(s_1, e_1, s_2, e_2, n):

```

1   $xor := s_1 \hat{ } s_2$ 
2   $first\_bit := \_builtin\_clz(xor)$ 
3   $\ell := first\_b$ 
4  return ( $\ell$ )

```

3 The data structure:

Introduction: Wild and Munro's paper shows no specific code for their stack data structure, this is probably due to the fact that they simply use a standard stack of height $\log n$. Thus, we will have no 'counterpart' algorithm to analyze. Our algorithm implements a modified version of a stack which we call "C-Bounded Stack"/"Forgetful Stack".

3.1 C-Bounded Stack:

Commentary on our data structure: We keep the beginning of the first run present in the array. This run will be the run that we will 'forget' once the array is full. Every time we forget a run, we will update the beginning of the entire structure as the end of the run we are forgetting + 1 (This changes with directions, need to revise it). This is due to the fact that runs are contiguous, so, the beginning of a run is exactly the ending of the previous run + 1. We only keep the beginning and end of the first run, alongside its power with the next 'found' run. (Given that we are going both directions, a.k.a scanning right and then left, each time we find a run, we calculate its power with the previously found run. In the algorithm, s1 is the previously found run, s2 is the newly found run, the power is calculated between both of those runs and then s1 gets into the stack alongside the power calculated.). Besides the information regarding the first run, we also keep the power and the end of every other run inserted in the stack. Top and Add algorithm are standard, I believe, although they're somewhat funky. We add an element directly on the top position (which is why to return the top element we need to do return top-1). Bottom keeps the first run inserted in the stack (the 'oldest'). RunIni keeps the beginning index of the run in bottom. MaxSize is the maximum capacity of the stack and size is the current amount of elements in the stack. Notice that we don't 'clean' our array as we pop elements, we just allow that position to be 'overwritten' through an add (this is probably standard practice too). The 'code space'/'code block' needs to be rewritten to look a little bit prettier. I'm going to separate each function in a specific block. The algorithm can be found in the next page.

Algorithm 4 C-Bounded Stack(initial run s_1 , size C):

```
1 top := 1
2 bottom := 1
3 size := 0
4 MaxSize := C
5 RunIni :=  $s_1$            // We keep the index of the beginning of the first run
6 array[MaxSize]
7 func Pop(){
8   if top == 1 and bottom > top:    // Is this second comparison necessary?
9     top ← MaxSize
10  else
11    top--
12  C-- }
13 func Add(run  $e_i$ , power k){
14  if C == MaxSize:                // i think theres a smarter way to do this
15    s1 ← array[bottom].end+1 // We need to update which run is the first
16    bottom ← (bottom + 1)%MaxHeight
17    array[top] ← ( $e_1$ , k)
18    top ← (top + 1)%MaxHeight
19  else
20    array[top] ← ( $e_1$ , k)
21    top ← (top + 1)%MaxHeight
22    C++
23  }
24 func Top(){
25  return array[(top == 1) ? MaxHeight : (top - 1)] }
```
